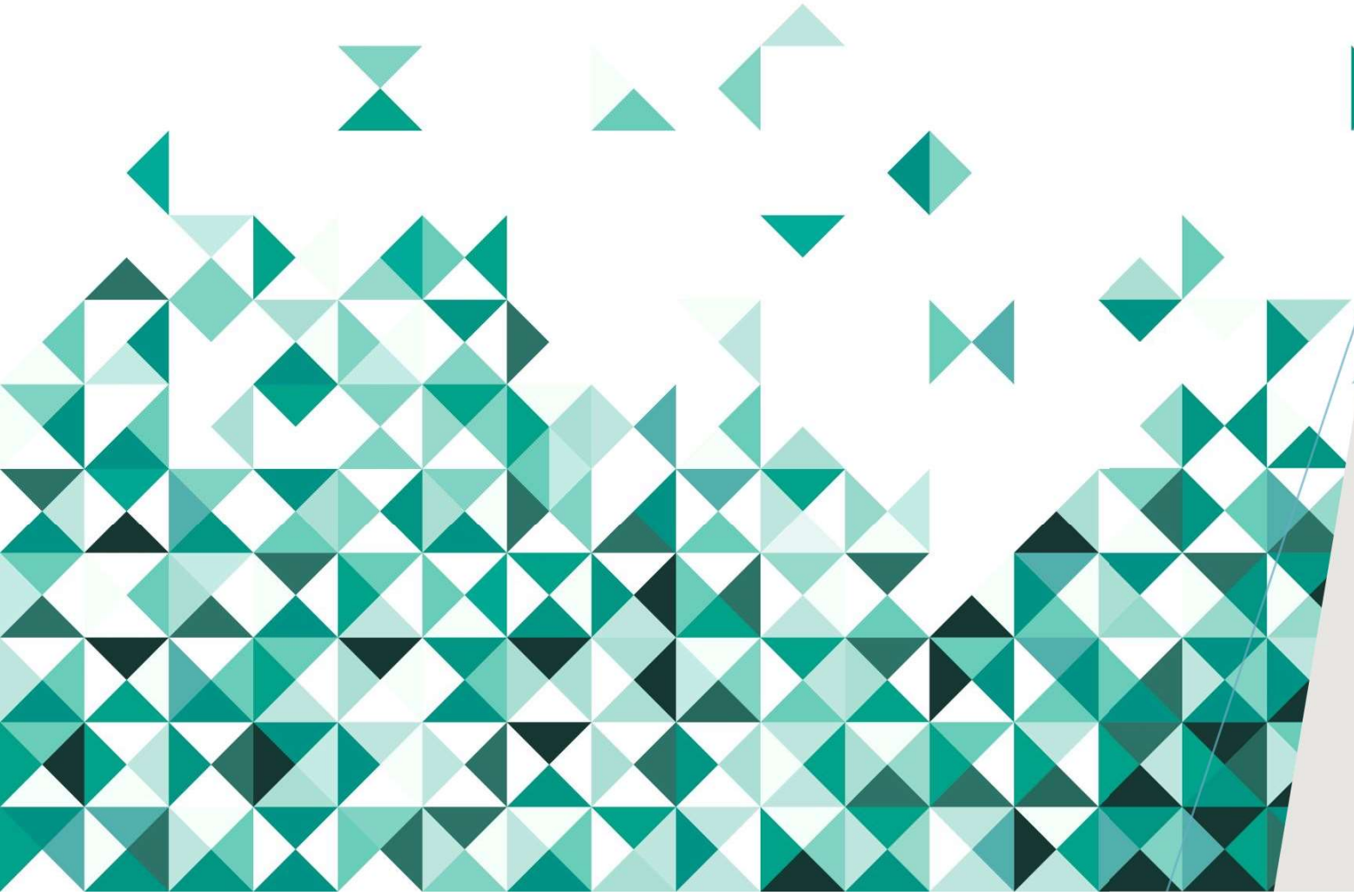


BCSE204L

DESIGN AND ANALYSIS OF ALGORITHMS

INSTRUCTOR DETAILS

- Dr. Ilanthenral Kandasamy
- Email: ilanthenral.k@vit.ac.in

COURSE OBJECTIVES

1. To provide mathematical foundations for analysing the complexity of the algorithms
2. To impart knowledge on various design strategies that can help in solving real-world problems effectively
3. To synthesise efficient algorithms in various engineering design situations

COURSE OUTCOMES

1. Apply the mathematical tools to analyze and derive the running time of the algorithms
2. Demonstrate the major algorithm design paradigms.
3. Explain major graph algorithms, string matching and geometric algorithms along with their analysis.
4. Articulating Randomized Algorithms.
5. Explain the hardness of real-world problems with respect to algorithmic efficiency and learning to cope with it.

M1: DESIGN PARADIGMS: GREEDY, DIVIDE AND CONQUER TECHNIQUES

- Overview and Importance of Algorithms - Stages of algorithm development: Describing the problem, Identifying a suitable technique,
- Design of an algorithm, Derive Time Complexity, Proof of Correctness of the algorithm,
- Illustration of Design Stages –
- Greedy techniques: Fractional Knapsack Problem and Huffman coding
- Divide and Conquer: Maximum Subarray, Karatsuba faster integer multiplication algorithm.

M2: DESIGN PARADIGMS: DYNAMIC PROGRAMMING, BACKTRACKING AND BRANCH & BOUND TECHNIQUES

- Dynamic programming:
  - Assembly Line Scheduling,
  - Matrix Chain Multiplication,
  - Longest Common Subsequence,
  - 0-1 Knapsack,
  - TSP
- Backtracking: N-Queens problem, Subset Sum, Graph Coloring
- Branch & Bound: LIFO-BB and FIFO BB methods: Job Selection problem, 0-1 Knapsack Problem

M3: STRING MATCHING ALGORITHMS

- Naïve String-matching Algorithms,
- KMP algorithm,
- Rabin-Karp Algorithm,
- Suffix Trees.

M4: GRAPH ALGORITHMS

- All pair shortest path:
  - Bellman Ford Algorithm,
  - Floyd-Warshall Algorithm
- Network Flows: Flow Networks,
- Maximum Flows: Ford-Fulkerson, Edmond-Karp, Push Re-label Algorithm
- Application of Max Flow to maximum matching problem

M5: GEOMETRIC ALGORITHMS

- Line Segments: Properties, Intersection, sweeping lines –
- Convex Hull finding algorithms: Graham's Scan, Jarvis' March Algorithm

M6: RANDOMIZED ALGORITHMS

- Randomized quick sort –
- The hiring problem –
- Finding the global Minimum Cut.

M7: CLASSES OF COMPLEXITY AND APPROXIMATION ALGORITHMS

- The Class P
- The Class NP
- Reducibility and NP-completeness
- SAT (Problem Definition and statement), 3SAT,
- Independent Set, Clique,
- Approximation Algorithm – Vertex Cover, Set Cover and Travelling salesman

M8: CONTEMPORARY ISSUES

- Recent issues

THEORY ASSESSMENT CONFIGURATION

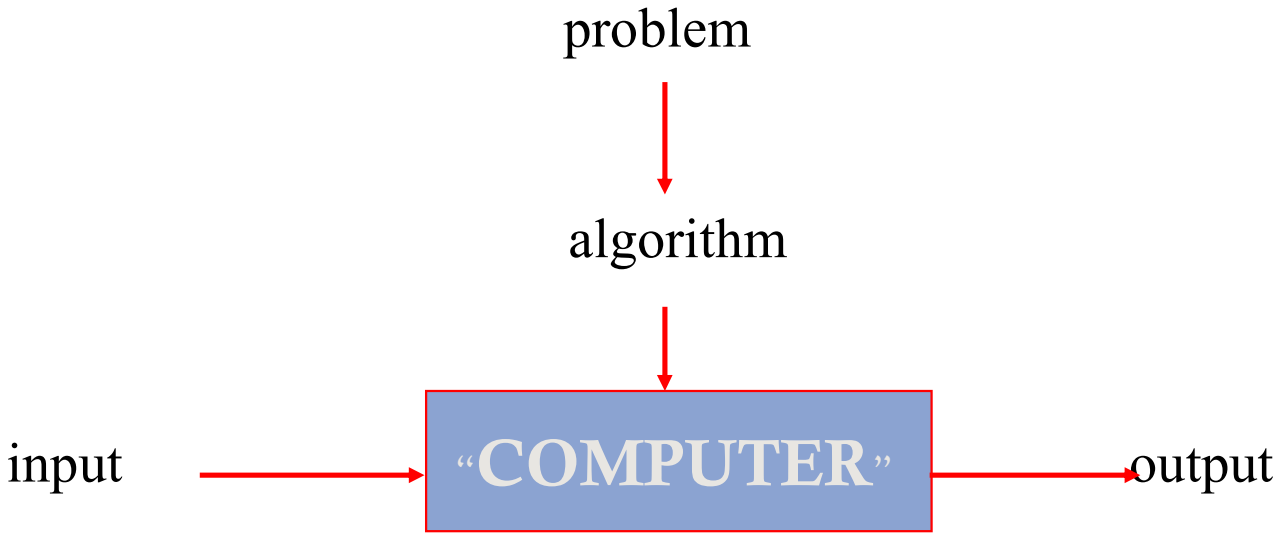
- DA – 10 marks - Continuous groups of assignments given
- Quiz 1 – 10 marks
- Quiz 2 – 10 marks
- CAT 1 – 15 marks
- CAT 2 – 15 marks
- FAT – 40 marks

LAB ASSESSMENT

- 4 DAs – 10 marks each
- Midterm – exam – 20 marks
- FAT – 40 marks

WHAT IS AN ALGORITHM?

An *algorithm* is a sequence of unambiguous instructions for solving a problem, i.e., for obtaining a required output for any legitimate input in a finite amount of time.



ALGORITHM

- An *algorithm* is a sequence of unambiguous instructions for solving a problem, i.e., for obtaining a required output for any legitimate input in a finite amount of time.

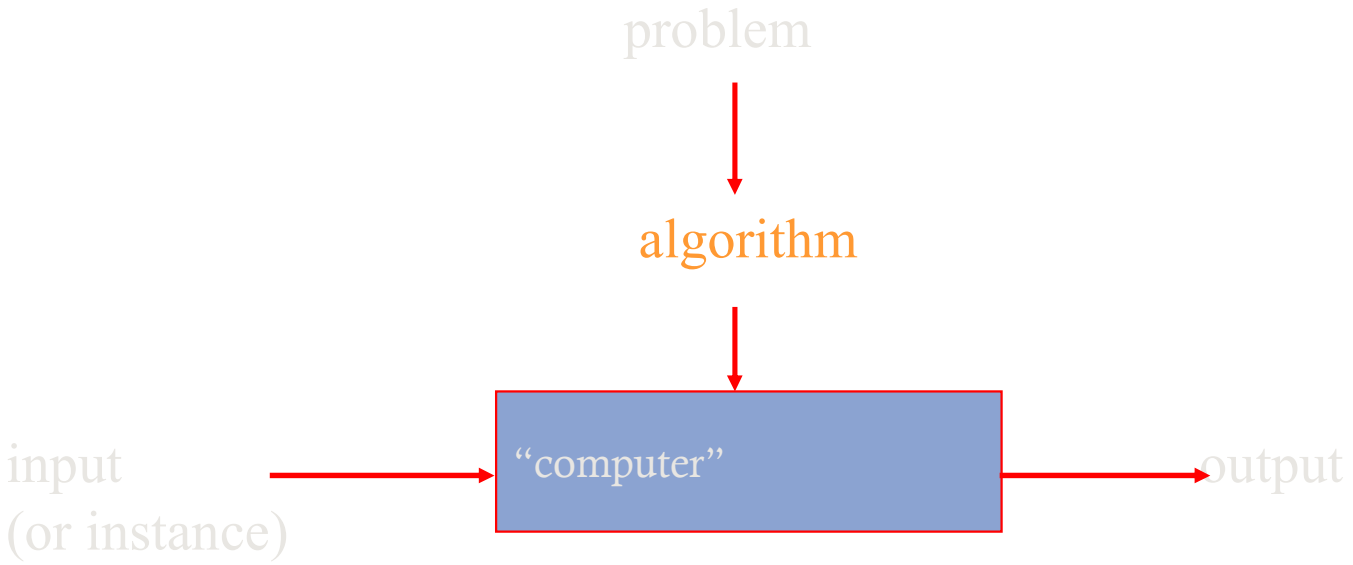
  - Can be represented various forms
  - Unambiguity/clearness
  - Effectiveness
  - Finiteness/termination
  - Correctness

HISTORICAL PERSPECTIVE

- Euclid's algorithm for finding the greatest common divisor

- Muhammad ibn Musa al-Khwarizmi – 9th century mathematician
  www.lib.virginia.edu/science/parshall/khwariz.html

NOTION OF ALGORITHM AND PROBLEM



algorithmic solution
(different from a conventional solution)

EXAMPLE OF COMPUTATIONAL PROBLEM: SORTING

- Statement of problem:
  - *Input:* A sequence of n numbers $\langle a_1, a_2, \ldots, a_n \rangle$
  - *Output:* A reordering of the input sequence $\langle a'_1, a'_2, \ldots, a'_n \rangle$ so that $a'_i \leq a'_j$ whenever $i < j$
- Instance: The sequence <5, 3, 2, 8, 3>
- Algorithms:
  - Selection sort
  - Insertion sort
  - Merge sort
  - (many others)

SELECTION SORT

- Input: array a[1], ⋯, a[n]

- Output: array a sorted in non-decreasing order
- Algorithm:

```
for i=1 to n
    swap a[i] with smallest of a[i], ⋯, a[n]
```

- Is this unambiguous? Effective?

SOME WELL-KNOWN COMPUTATIONAL PROBLEMS

- Sorting
- Searching
- Shortest paths in a graph
- Minimum spanning tree
- Primality testing
- Traveling salesman problem
- Knapsack problem
- Chess
- Towers of Hanoi
- Program termination

Some of these problems don't have efficient algorithms, or algorithms at all!

BASIC ISSUES RELATED TO ALGORITHMS

- How to design algorithms
- How to express algorithms
- Proving correctness
- Efficiency (or complexity) analysis
  - Theoretical analysis
  - Empirical analysis
- Optimality

ALGORITHM DESIGN STRATEGIES

- Brute force
- Divide and conquer
- Decrease and conquer
- Transform and conquer

- Greedy approach
- Dynamic programming
- Backtracking and branch-and-bound
- Space and time tradeoffs

ANALYSIS OF ALGORITHMS

- How good is the algorithm?
  - Correctness
  - Time efficiency
  - Space efficiency

- Does there exist a better algorithm?
  - Lower bounds
  - Optimality

WHAT IS AN ALGORITHM?

- Recipe, process, method, technique, procedure, routine,… with the following requirements:

1. Finiteness
   - terminates after a finite number of steps
2. Definiteness
   - rigorously and unambiguously specified
3. Clearly specified input
   - valid inputs are clearly specified
4. Clearly specified/expected output
   - can be proved to produce the correct output given a valid input
5. Effectiveness
   - steps are sufficiently simple and basic

WHY STUDY ALGORITHMS?

- Theoretical importance
  - the core of computer science
- Practical importance
  - A practitioner's toolkit of known algorithms
  - Framework for designing and analyzing algorithms for new problems

Example: Google's PageRank Technology

EUCLID'S ALGORITHM

Problem: Find gcd(m,n), the greatest common divisor of two nonnegative, not both zero integers m and n

Examples: gcd(60,24) = 12, gcd(60,0) = 60, gcd(0,0) = ?

Euclid's algorithm is based on repeated application of equality

$$\gcd(m,n) = \gcd(n, m \bmod n)$$

until the second number becomes 0, which makes the problem trivial.

Example: gcd(60,24) = gcd(24,12) = gcd(12,0) = 12

TWO DESCRIPTIONS OF EUCLID'S ALGORITHM

Step 1 If $n = 0$, return m and stop; otherwise go to Step 2

Step 2 Divide m by n and assign the value of the remainder to r

Step 3 Assign the value of n to m and the value of r to n. Go to Step 1.

while $n \neq 0$ do

$r \leftarrow m \bmod n$

$m \leftarrow n$

$n \leftarrow r$

return m

OTHER METHODS FOR COMPUTING GCD(M,N)

Consecutive integer checking algorithm

Step 1 Assign the value of min{*m*,*n*} to *t*

Step 2 Divide *m* by *t*. If the remainder is 0, go to Step 3; otherwise, go to Step 4

Step 3 Divide *n* by *t*. If the remainder is 0, return *t* and stop; otherwise, go to Step 4

Step 4 Decrease *t* by 1 and go to Step 2

Is this slower than Euclid's algorithm?

How much slower?

O(n), if n <= m , vs O(log n)

OTHER METHODS FOR GCD(M,N) [CONT.]

Middle-school procedure

Step 1 Find the prime factorization of *m*

Step 2 Find the prime factorization of *n*

Step 3 Find all the common prime factors

Step 4 Compute the product of all the common prime factors and return it as gcd*(m,n)*

Is this an algorithm?

How efficient is it? Time complexity: O(sqrt(n))

GREATEST COMMON DIVISOR (GCD)

- GCD (a,b) of a and b is the largest integer that divides evenly into both a and b
  - eg GCD(60,24) = 12,
  - **Find GCD(1970,1066)**
- define gcd(0, 0) = 0
- often want **no common factors** (except 1) define such numbers as **relatively prime**
  - eg GCD(8,15) = 1
  - hence 8 & 15 are relatively prime

EXAMPLE GCD(1970,1066)

· 1970 = 1 x 1066 + 904	gcd(1066, 904)
· 1066 = 1 x 904 + 162	gcd(904, 162)
· 904 = 5 x 162 + 94	gcd(162, 94)
· 162 = 1 x 94 + 68	gcd(94, 68)
· 94 = 1 x 68 + 26	gcd(68, 26)
· 68 = 2 x 26 + 16	gcd(26, 16)
· 26 = 1 x 16 + 10	gcd(16, 10)
· 16 = 1 x 10 + 6	gcd(10, 6)
· 10 = 1 x 6 + 4	gcd(6, 4)
· 6 = 1 x 4 + 2	gcd(4, 2)
· 4 = 2 x 2 + 0	gcd(2, 0)

TWO MAIN ISSUES RELATED TO ALGORITHMS

- How to design algorithms

- How to analyze algorithm efficiency

ALGORITHM DESIGN TECHNIQUES/STRATEGIES

- Brute force
- Divide and conquer
- Decrease and conquer
- Transform and conquer
- Space and time tradeoffs

- Greedy approach
- Dynamic programming
- Iterative improvement
- Backtracking
- Branch and bound

ANALYSIS OF ALGORITHMS

- How good is the algorithm?
  - time efficiency
  - space efficiency
  - correctness ignored in this course

- Does there exist a better algorithm?
  - lower bounds
  - optimality

IMPORTANT PROBLEM TYPES

- sorting
- searching
- string processing
- graph problems
- combinatorial problems
- geometric problems
- numerical problems

ANALYSIS OF ALGORITHM

PROOF OF CORRECTNESS

- The algorithm is complete/correct: the post-condition is respected on all possible inputs satisfying the pre-condition
- Precondition: a predicate I on the input data
- Postcondition: a predicate O on the output data
- Correctness: proving $I \Rightarrow O$

- The algorithm terminates: For all possible input, the algorithm exits

LOOP INVARIANTS

- *A loop invariant is a logic formula or just a set of rules, that is true before, during and after the loop in question (so it's unbiased to iteration). It is imperative for it to contain rules for all the variables that occur in the said loop because we need to **bind** them all to the set of values we want them to be.*
- A loop invariant can be as complicated and as simple as you want it to be. However, the point is that it should be constructed to resemble the problem at hand as closely as possible.

LOOP INVARIANTS

- **Initialization:** It is true prior to the first iteration of the loop.
- **Maintenance:** If it is true before an iteration of the loop, it remains true before the next iteration
- **Termination:** When the loop terminates, the invariant gives us a useful property that helps show that the algorithm is correct.

EXAMPLE

```
x = 10
y = 4
z = 0
n = 0
while(n < x):
        z = z+y
        n = n+1
```

EXAMPLE

- Logically, this code just calculates the value x*y and stores it in z, this means that z = x*y.
- z = n*y

BEFORE LOOP

x = 5

y = 4

z = 0

n = 0

- RULE 1: z == n*y

  0 == 0*4 = 0 <=> True

  so RULE 1 applies

MAINTENANCE

x = 5, y = 4 while (0 < 5) z = 0+4; n =0 +1;

So z` = 4

So n`= 1

After

- RULE 1: z’ == n*y

  4 == 1*4 = 4 <=> True

  so RULE 1 applies

#1

MAINTENANCE

#2

x = 5, y = 4

z` = 8

N`= 2

After

- RULE 1: z == n*y

  8 == 2*4 = 8 <=> True

  so RULE 1 applies

MAINTENANCE

Iterations ….. Contd.

TERMINATION

N`= 5 while (5 < 5) loop fails, so n and z wont change

- RULE 1: z == n*y

  20 == 5*4 = 20 <=> True

  so RULE 1 applies

INSERTION SORT

INSERTION-SORT(A)

```
1  for j = 2 to A:length
2          key = A[j]
3          // Insert A[j]  into the sorted sequence A[1 .. j – 1].
4          i = j - 1
5          while i > 0 and A [i] > key
6                  A[i+1] = A[i]
7                  i = i - 1
8          A[i+1] = key
```

LOOP INVARIANT

- A[1. . j – 1] constitutes the currently sorted set.
- A[j+1. . N] corresponds to the remaining elements in array.

- Loop invariant : At the start of each iteration of the **for** loop of lines 1–8, the subarray A[1. . j – 1] consists of the elements originally in A[1. . j – 1], but in sorted order.

INITIALIZATION

- **Initialization:** We start by showing that the loop invariant holds before the first loop iteration, when $j = 2$. The subarray $A[1..j-1]$, therefore, consists of just the single element $A[1]$, which is in fact the original element in $A[1]$.
- Moreover, this subarray is sorted, which shows that the loop invariant holds prior to the first iteration of the loop.

MAINTENANCE

- **Maintenance:** Each iteration maintains the loop invariant.
- The body of the **for** loop works by moving A[j – 1], A[j – 2], A[j – 3], and so on by one position to the right until it finds the proper position for A[j] (lines 4–7), at which point it inserts the value of A[j] (line 8).
- The subarray A[1 . . J] then consists of the elements originally in A[1 . . J] , but in sorted order.
- Incrementing j for the next iteration of the **for** loop then preserves the loop invariant.

TERMINATION

- **Termination:** Finally, we examine what happens when the loop terminates.
- The condition causing the **for** loop to terminate is that j >A.*length* = n. Because each loop iteration increases j by 1.
- The subarray A[1 .. n] consists of the elements originally in A[1 . . N], but in sorted order.
- Observing that the subarray A[1 . . n] is the entire array, we conclude that the entire array is sorted.
- Hence, the algorithm is correct.